[Fall 2025] ROB-GY 6203 Robot Perception Homework 1

Aadhav Sivakaumar

Submission Deadline (No late submission): NYC Time 5:00 PM, October 09, 2025

General Rules

1. Please submit the **.pdf** generated by this LaTex file. This .pdf file will be the main document for us to grade your homework. If you wrote any code, please zip all the **code** together and **submit a single .zip file**. Name the code scripts clearly or/and make explicit reference in your written answers. Do NOT submit very large data files along with your code!
2. This Overleaf project is view-only. To work on this document, please use the download feature on the top left and download the source files. Then, you can re-upload them to your own Overleaf account or any other LaTeX editing tools of your preference.
3. Please typeset your report in LaTeX/Overleaf. If you have never used LaTeX/Overleaf before, the best advice is to just start typing. Try things hands-on like a true engineer! If you prefer a more structured introduction, this video can be a good help. **Do NOT submit a hand-written report!** as they will be graded zero.

   LaTeX is the standard tool for STEM academic writing due to its convenience in typesetting mathematical formulas, so this class will encourage you to get familiar with this widely used tool.
4. Do not forget to update the variables "yourName" and "yourNetID".

Hints

1. While the HW may talk about AprilTag, you don't have to use it. You can use similar, but more accessible options, such as OpenCV's ArUco tag.
2. You don't have to print out a tag physically. Displaying them on a screen, such as your phone or iPad, would work most of the time. Make sure the background of the tag is white. In my experience, a tag on a black background is harder to detect.

Contents

Task 1 Sherlock's Message (2pt)

Detective Sherlock left a message for his assistant, Dr. Watson, while tracking his arch-enemy, Professor Moriarty. Could you help Dr. Watson decode this message? The original image itself can be found in the data folder of the Overleaf project, named `for_watson.png`

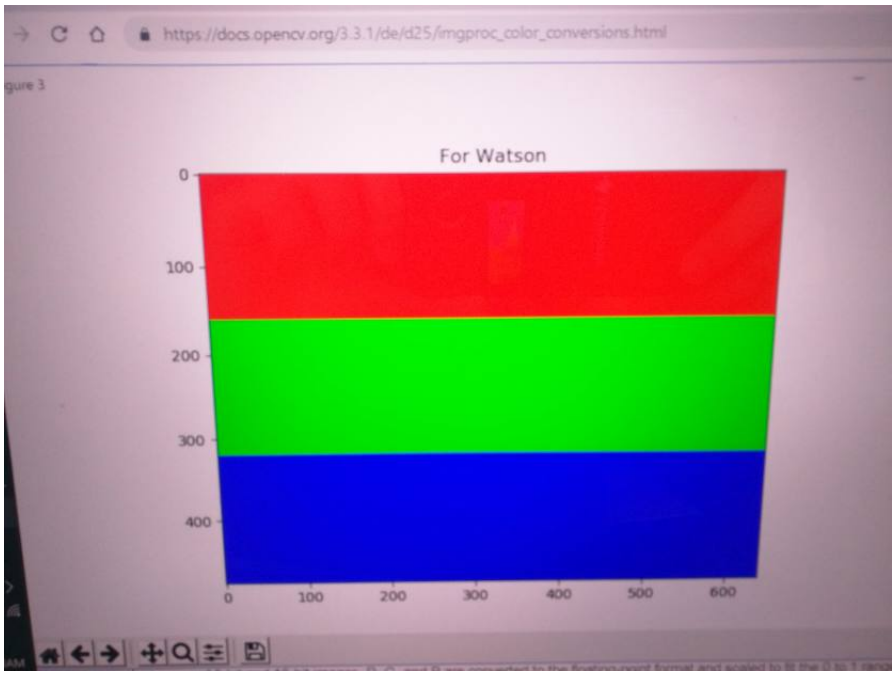
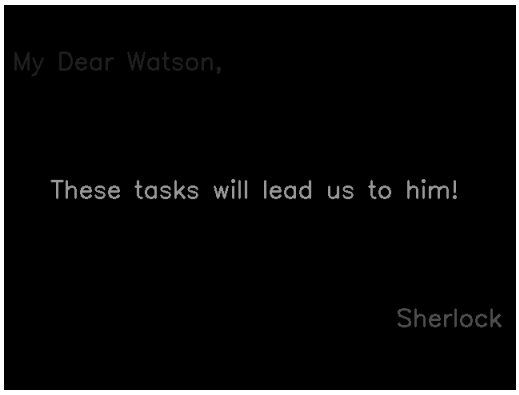


Figure 1: The Secret Message Left by Detective Sherlock

a) (1pt)

Please submit the image(s) after decoding. The image(s) should have the secret message on it(them). Screenshots or images saved by OpenCV is fine.

Answers:



b) (1pt)

Please describe what you did with the image with words, and tell us where to find the code you wrote for this question.

Answers:
The code is included in the zip file under "Task1.py". I decided to turn the image to grayscale because it's much easier to deal with and small differences are much more noticible. I saw that in the top section, there were mainly color values of 29. In the middle section, the grayscale value was 150. In the bottom section, the grayscale value was mainly 76. I iteratively removed all of the colors of those values and the only things that were left were pixels that were of slightly different values, which I noticed by printing out the full 2 dimensional 640x480 array representing the picture. In the top, middle, and bottom section respectively, there is text with the color values of 31, 151, and 77, just one or two grayscale values off from the rest of the section. I set the values of the sections that weren't part of the text to zero, and the message revealed itself.

Task 2. Low Dimensional Projection (5pt)

Given the Fasion-MNIST dataset, **train** an unsupervised learning neural network that gives you a lower-dimensional representation of the images, after which you could easily use tSNE from `Scikit-Learn` to bring the dimension down to 2. **Visualize** the results of all 10000 images in one single visualization.

Answers:

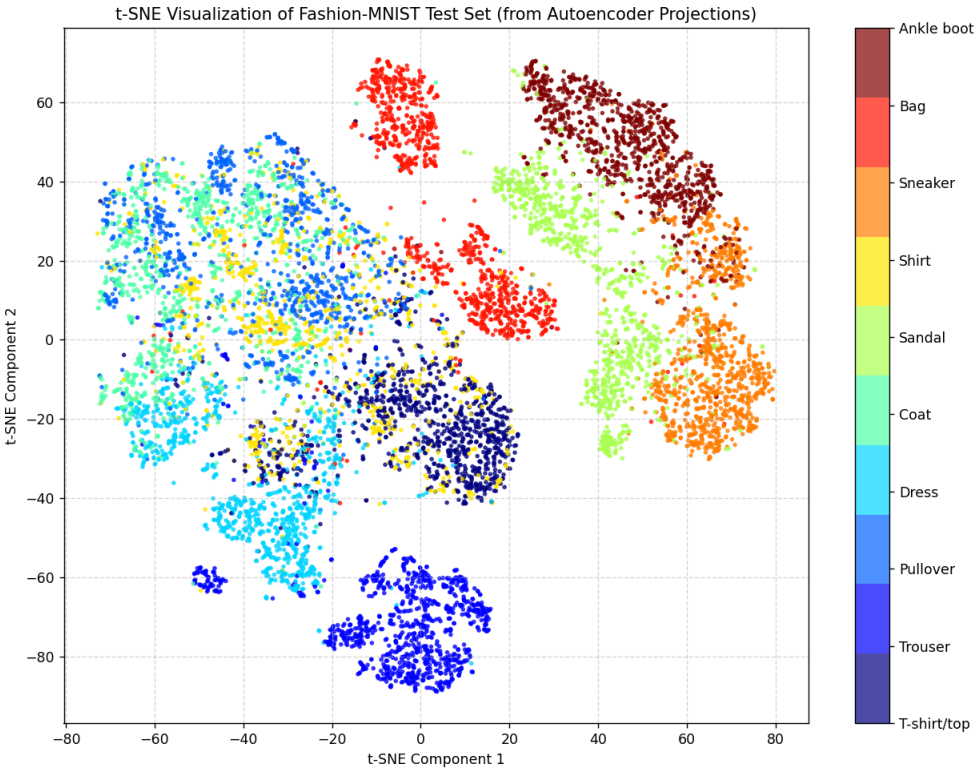


Figure 2: 2 dimensional visualization

```
EPOCHS = 50
BATCH_SIZE = 32
LEARNING_RATE = 2e-3
LATENT_DIM = 32
```

```
[t-SNE] Mean sigma: 1.138879
[t-SNE] KL divergence after 250 iterations with early exaggeration: 81.331337
[t-SNE] KL divergence after 1000 iterations: 1.735143
```

Figure 3: Hyperparameters and stats

```
Epoch [1/50], Average Loss: 0.0236
Epoch [2/50], Average Loss: 0.0153
Epoch [3/50], Average Loss: 0.0136
Epoch [4/50], Average Loss: 0.0126
Epoch [5/50], Average Loss: 0.0120
```

```
Epoch [46/50], Average Loss: 0.0087
Epoch [47/50], Average Loss: 0.0086
Epoch [48/50], Average Loss: 0.0086
Epoch [49/50], Average Loss: 0.0086
Epoch [50/50], Average Loss: 0.0086
Training complete.
```

Figure 4: Average loss at the start and end

Task 3 Camera Calibration (3pt)

Use the pyAprilTag package provided in the class, or other free packages (e.g., OpenCV's camera calibration toolkit) that you may be aware of, to **calibrate** your camera with a tag-based method and provide the full K matrix, along with the top two distortion parameters..

Answers:

```
----------------------------------------
Camera Intrinsic Matrix (K):
[[1.59701e+03 0.00000e+00 5.79710e+02]
 [0.00000e+00 1.62480e+03 3.37620e+02]
 [0.00000e+00 0.00000e+00 1.00000e+00]]

Distortion Parameters [k1, k2, p1, p2, k3]:
[[-6.57824627e+00  1.31725081e+02  1.25794000e-01 -4.32493244e-02
  -8.23916833e+02]]

Top two distortion parameters:
k1 = -6.5782
k2 = 131.7251
----------------------------------------
```

Figure 5: camera calibration matrix and parameters

Task 4 Tag-based Augmented Reality (5pt)

Use the pyAprilTag package to detect an AprilTag in an image (or use OpenCV for an Aruco Tag), for which you should take a photo of a tag. Use the K matrix you obtained above, to draw a 3D cube of the same size of the tag on the image, as if this virtual cube really is on top of the tag. **Document** the methods you use, and **show** your AR results from at least two different perspectives.

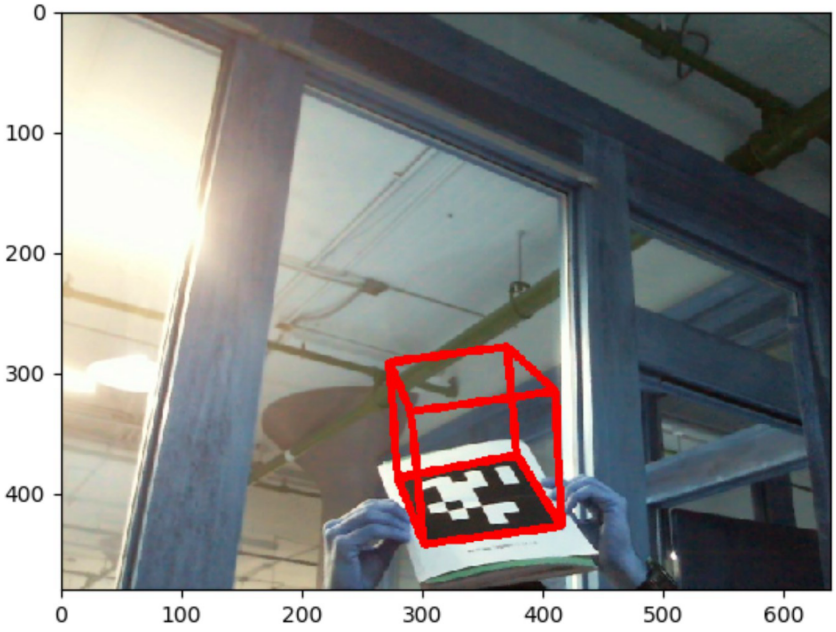


Figure 6: Caption

Tips: There are many ways to do this, but you may find OpenCV's `projectPoints`, `drawContours`, `addWeighted` and `line` methods useful. You don't have to use all these methods.

Answers:

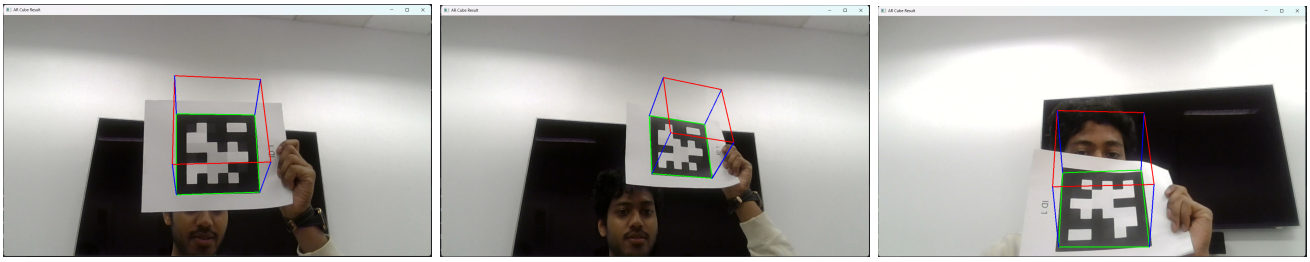


Figure 7: AR results from different perspectives

I used the camera calibration matrix that I got from the previous problem to work on this task. I also used inherent information (like the fact that the tag I printed is 15cmx15cm) to drive some of the detection code. It starts by first reading the image from opencv, and converting it to grayscale, for the same reason I mentioned in task 1. I then used the Arucodetector, using the Apriltag dictionary as an input to get the corners, and ids. Using the detected id, I can get the orientation of the tag by using the solvePnP function in cv2, where the camera calibration k matrix can be used as an input. From here, I can use drawContours to draw a green box around the tag, blue lines to denote the direction the tag is facing, and a red box to project the tag in order to make a cube.